

Tonabnehmerwickelmaschine

Inhalt

Einführung – Einen Tonabnehmer wickeln	2
Los Geht's	2
Hardwarekonfiguration	3
Ziele des Projektes	3
Was die Maschine können soll	3
Was die Maschine noch nicht können soll / evtl für später	3
Unklar	4
Achtung Aufpassen	4
Sonstiges	4
Organisation	4
ToDo	4
Inspirationen/Quellen	4
Open Source Projekte	5
Material- / Einkaufsliste	6
Dokumentation der Software	9
Konventionen	9
Schaltplan	9
Die Motoren programmieren	9
BLDC Motor (= WIND Motor)	9
Stepper Motor	11
Programmablauf	12
Setup	12
Loop	13
Emergency Stop	14
Funktionen im Überblick	14
calculateXAxisRowCount	14
calculateStepsToDistance	14
moveDistance	14

setMoveDirection	14
moveStepperPerTimeStep()	15
goToZero	15
moveToStart	15
Konfigurationseinstellungen – Config.h	15
User Calibrated Settings	15
Hardware Configuration	15
X-Axis Protocol	16
Hardware	16

Einführung – Einen Tonabnehmer wickeln

Los Geht's

Bevor gewickelt werden kann, müssen alle Settings in [Config.h](#) angepasst werden. Was die einzelnen Konstanten bedeuten ist in der Datei selbst, bzw. in der Dokumentation geschrieben.

Man nimmt einen Tonabnehmer und befestigt den Draht. Danach schraubt man ihn auf der Wickelplatte fest. Hier darauf achten, dass der Draht dabei durch die Kugelumlenkrollen der Drahtführung geht.

Die Stromversorgung an die Steckdose anschalten und auf den Arduino das Programm laden. Sollte das schonmal gemacht worden sein, kann man einfach den RESET Knopf auf dem Arduino drücken und er startet das letzte Programm.

Hier wird die Maschine sich Nullen, zur Startposition fahren und warten. Die Startposition kann in der Config.h Datei angepasst und eingestellt werden. Sobald der START Taster gedrückt wird, beginnt die Maschine den Tonabnehmer zu wickeln.



Hardwarekonfiguration

Es empfiehlt sich, die Wind Platte so nah wie möglich an die X-Achse heranzuschieben. Dadurch wackelt der Draht nicht so stark in der Luft und die Achsenführung ist präziser.

Die Drahttrommel sollte auf dem Boden unter der Apparatur befestigt bzw. hingestellt werden. Es ist für den Draht viel leichter sich so abzuwickeln.

Der Filz sollte generell mäßig fest zugezogen werden, sodass der Draht auf Spannung ist aber nicht reißt. Die Spannung ist wichtig, weil dadurch fällt er besser in seine Bahn und die einzelnen Umwicklungen sind fester am Tonabnehmer.

Bei den Tonabnehmern dürfen auch keine Schrauben heraus schauen, sonst wickelt sich der Draht dort ja drumherum.

Ziele des Projektes

Was die Maschine können soll

- Bedienung:
 - Computer: spielt per USB Kabel auf Arduino / Raspberry Pi eine Einstellung für die Umdrehungsgeschwindigkeit, Anzahl der Wicklungen, Richtung der Umdrehung???
 - Potis und Schalter ohne Display, aber mit manuellen Markierungen für Werte
- Funktionsweise:
 - Komplette automatisch: man steckt die Spule und leere Tonabnehmerbox auf einen Halter, gibt die Einstellungen an die Maschine weiter und bei Druck auf Startknopf geht's los
 - LED als Statusanzeige
 - Arduino/ Raspberry Pi / ESP32 als Steuerung
- Skalierung:
 - Auf jeden Fall modular, sodass die Maschine leicht ausbaubar und reparierbar ist.
 - Maschine lieber teurer, aber dafür lange haltbar und man nicht in einem Jahr eine neue bauen muss / ständig verbessern muss (vgl. CNC)
- Kosten: maximal 500€ und 4 Wochen Zeitaufwand, aber erstmal Kostenliste erstellen
- Benutzerhandbuch??? Bzw. kleines Hilfsblatt

Was die Maschine noch nicht können soll / evtl für später

- Eigenes Display, wo Sachen eingestellt werden (mit Potis, Schaltern)
- System zum Wachsen (mit Haltern, bei denen unten drunter Auffangbecken für Wachs ist, Zeitstopper) → Eigene Maschine, hat nichts mit Wickeln direkt zu tun, außerdem unklar inwiefern Wachsen wichtig ist.
- Benachrichtigungssystem, dass man eine Nachricht aufs Handy bekommt, wann die Maschine fertig ist (App, womit man die Maschine auch steuern kann???)

Unklar

- Wie wird der Tonabnehmer aufgebaut sein? Zwei Platten, die verschraubt werden, oder ein 3D-gedrucktes teil? -> wichtig

Achtung Aufpassen

- Scheibe darf nicht eiern, aus Metall, stabile Führung zum Motor
- Fürs Display Ports am Arduino/Esp übriglassen.
- Magnetisierer nach dem wickeln
- Wachsbad
- Ebay / Kleinanzeigen schauen
- Open Source schauen

Sonstiges

- Bei Tension ab 35gr -> stoppt Maschine von Tonewinder.es
- Es wäre genial, wenn man überprüfen kann, ab wann man einen Riss erkennt und die Maschine noch so schnell davor stoppt, dass der Draht nicht reißt.

Organisation

ToDo

1. Quellen anschauen mit dem Ziel
2. gutes Design für die Wickelmaschine zu entwerfen.
3. Bauteile raussuchen, Liste mit Links und Kostenübersicht erstellen

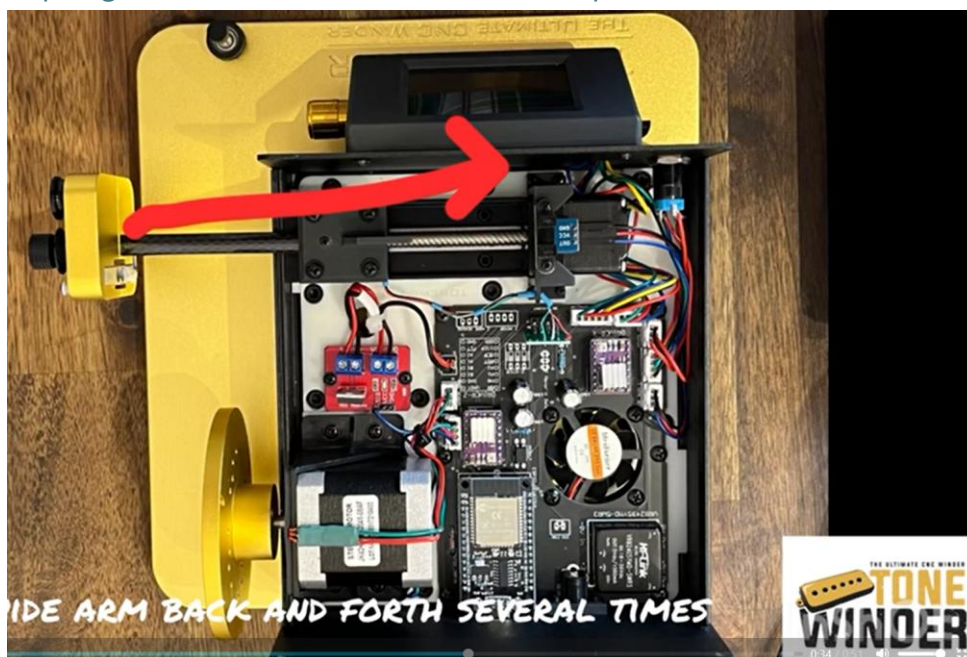
Inspirationen/Quellen

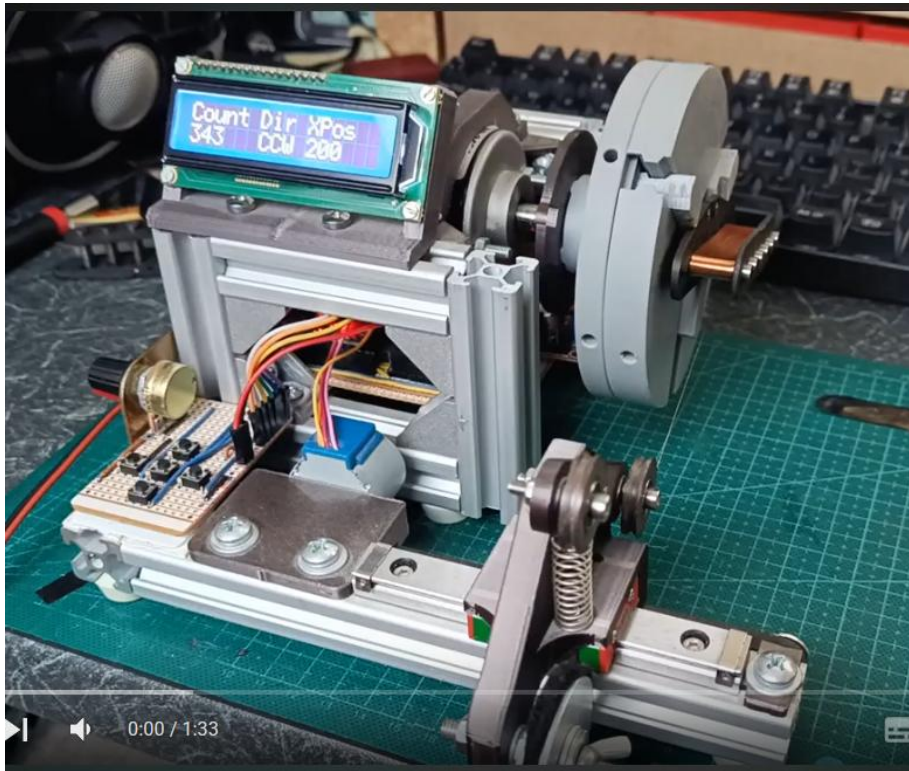
- <https://www.youtube.com/watch?v=KPvIYBYeZ7k>
- https://www.youtube.com/results?search_query=coil+winding+machine
- <https://www.youtube.com/watch?v=BNMbEHGXCMY>
- <https://www.youtube.com/watch?v=n5D-e8fWkMQ>
- <https://www.youtube.com/watch?v=h0zzOOvF474>
- <https://www.stewmac.com/luthier-tools-and-supplies/tools-by-job/tools-for-electronics/pickup-building/schatten-pro-pickup-winder>
- <https://www.stewmac.com/luthier-tools-and-supplies/tools-by-job/tools-for-electronics/pickup-building/>
- <https://www.youtube.com/watch?v=5JMsRX6SGlw>
- <https://www.youtube.com/watch?v=5BqFoBJr10o>
- <https://tonewinder.es/#home>
- <https://www.youtube.com/watch?v=VwM77fmqV7s> (Measure Pickups)

- https://wynnmechatronics.com/auto_traverse/
- <https://youtu.be/xya4ipu7bTk?si=i8tGiBE1dEXlZ-2V> (DIY Pickup – Mr Innovative)
- <https://www.youtube.com/watch?v=Kz45yB57cc0>
- <https://www.exsmokers.eu/pages/wickelrechner> (wickelrechner)
- <https://www.nanotec.com/eu/de>
- <https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/> (l298n)
- <https://www.youtube.com/watch?v=KXmSkIBSp8E> (Motor + Driver)
- <https://www.konstruktionsatlas.de/antriebstechnik/welle-nabe-verbindung-formschluss.shtml>
- <https://ww3.cad.de/foren/ubb/Forum200/HTML/000574.shtml>
- <https://howtomechatronics.com/tutorials/arduino/stepper-motors-and-arduino-the-ultimate-guide/> (Stepper-Motor)

Open Source Projekte

- <https://www.bogdanbacila.com/posts/hobby/open-pickup-winder/>
- https://bojit.org/projects/Pickup_Winder/
- <https://www.thingiverse.com/thing:6760322>
- <https://github.com/fps/OPWF>
- <https://www.tdpri.com/threads/diy-pickup-winder-project.1169336/>
- https://github.com/AnotherBuildCorner/Pickup_Winder/tree/main





Material- / Einkaufsliste

Sind keine Affiliate Links...

Was	Zweck	Preis/ Stück	Anzahl	Link	Status
Motor	Umdrehungen	53€	1	https://ebay.us/m/u4SWYX (4000 U/min, mit driver)	angekommen
Motorhalterung	Nema 17	2,5€	4	https://amzn.eu/d/8ra9CXA	angekommen
Motor (NEMA17)	Kabelführung (1A / Phase) Wir haben Nema 23 + Motordriver	10€	1	https://amzn.eu/d/3cFvccM	Auf Lager
Motordriver			1		Beim Motor enthalten
Umlaufrollen (groß)	Kabelführung	0,76€	10	https://amzn.eu/d/1BoLx2Z	angekommen
U-Profil + T-Profil / Winkel	Halterung Abreissystem	3,5€	1	https://www.aluminium-online-shop.de/produkt-kategorie/aluminium-profile/u-profile/ https://www.profilzuschnitt24.de/t-profil-alu-40x40x4mm.html	Recherche
Tension Screw	Um etwas zug einzustellen (selbst bauen)	9€	1	https://amzn.eu/d/7nUEyR Schraube darauf Beilagscheiben mit mutter. Filz zwischen den Beilagscheiben	Eigenbau, auf Lager

	(zwei Beilagscheiben mit Filz dazwischen)				
Arduino Uno	Steuerung Ursprünglich: Esp32, aber weil für Motordriver Steuerungsspannung >3,5V gebraucht wird -> Uno		1		Auf Lager
WiskerDisk	Gleichmäßige Kraftverteilung auf Spule beim abwickeln bauen wir uns selbst aus angeldraht und holz/Pappe	selbst bau	1	https://groupdiy.com/threads/wisker-disk-wire-tensioner-diy.56712/ (DIY) https://youtu.be/OjV_PAQqYB8 https://youtu.be/GstWWMzUxc	Eigenbau
Linearführung	Kabelführung		1		Im Linearmodul
Gewindestange	Kabelführung (250mm)		1		Im Linearmodul
Magnetsensor	Umdrehungsmessung		1		Hallsensor im BLDC Motor enthalten
Breadboard / Lochplatine	Elektronikschaltung	2,66€	3	https://amzn.eu/d/hN84WQB	Auf Lager
Kabel	Elektronikverknüpfung	7€	120	https://amzn.eu/d/bUq3YKw	Auf Lager
Bundle: Linearmodul	Gewindestange, Linearführung. 150mm	82€	1	https://ebay.us/m/vjl4PC	Angekommen
Holz / Metallquader	Wird die Tonabnehmerhülse erstgemacht				Auf Lager
Aluprofile	Gestell				Auf Lager
Wellenkupplung	Windungssystem			https://amzn.eu/d/1JrsCBr https://amzn.eu/d/cCtYFjp	Im Linearmodul
Alurundscheibe				https://ebay.us/m/JQlrV7 https://ebay.us/m/NkbLqS	Erstmal 3d-gedruckt
Planscheibe / Frontscheibe einer		10€		https://amzn.eu/d/eEb7EAL https://amzn.eu/d/1oSjNS9	Erstmal 3d gedruckt

Drehmaschine Polier- und Schleifscheibe aus Aluminium für flache Maschinen/ Flexscheiben gekröpft				https://de.aliexpress.com/item/32291529571.html?spm=a2g0o.tesla.0.0.238bYeWwYeWwSU&pdp_npi=6%40dis%21EUR%2116%2C18%E2%82%AC%219%2C69%E2%82%AC%21%21%21%21%21%40211b813b17576811329196749edfc9%2112000021787067789%21btf%21%21%21%211%210%21&afTraceInfo=32291529571_pc_c_ppc_item_bridge_pc_jfy_wf_D4yuXKX_1757681133167&gatewayAdapt=glo2deu https://amzn.eu/d/2ENZ88s https://amzn.eu/d/4ooPUeE https://amzn.eu/d/40bAGka	
Schraubenau fsatz akkuflex		5€		https://amzn.eu/d/9VV705x	Erstmal 3d gedruckt
Flanschkuppl ung		5€ 10€		https://amzn.eu/d/eq3T3Jx https://amzn.eu/d/6RJtldM	Erstmal 3d gedruckt
Draht	Anscheinend schlecht zu löten, d.h. wenn draht reißt ist schlecht.	10€	1500m	https://www.conrad.de/de/p/tru-components-kupferlackdraht-aussen-durchmesser-inkl-isolierlack-0-05-mm-1500-m-1570224.html	
Kupferlackdra ht		29,80 €	9000m	https://www.rockinger.com/werkstatt/pickup-winding/1198/rockinger-premium-pickup-wickeldraht-0-063mm?number=07091	angekomme n
Tonabnehmer wickelset		25,90 €		https://www.rockinger.com/werkstatt/pickup-winding/1205/pickup-winding-kit-fuer-strat	Erstmal 3d gedruckt
MDF Platte		Ca 20€	40cmx 40cm	Baumarkt, zum festschrauben der ganzen Alu Profile	angekomme n
Drehlager		6,99€		https://amzn.eu/d/bu759lb	bestellt
DC DC StepDown converter		9,50€	5	https://amzn.eu/d/0IK5Czr	angekomme n
Power Supply		22€	1	https://amzn.eu/d/9aV1HqZ	angekomme n
Taster	Zum Nullen der Achse	4,60€	20	https://amzn.eu/d/88HliOT	angekomme n

Gesamtkosten: 187 € (ohne Drucksensor, Aluprofile, Draht selbst, Tonabnehmerhülsen, Magnete für Tonabnehmer, Drehlager)

Dokumentation der Software

Konventionen

Source Code des Arduinos wird in Zitat-Blöcken eingebettet:

```
| int x = 42;    // Beispiel Codeblock
```

Schaltplan

Siehe: Tonabnehmerwickelmaschine_Schaltplan.pdf

Der Powersupply wird einfach an die Steckdose angeschlossen. Ich schätze da ist ein Trafo drinnen und das ist somit galvanisch getrennt vom Rest der Schaltung. Deshalb steht das auch im Schaltplan. Da bin ich mir aber nicht sicher.

Die Motoren programmieren

BLDC Motor (= WIND Motor)

Der BLDC Motor wird im Code auch als WIND Motor oder WIND bezeichnet.

RPM des BLDC Motors messen

Wie viele Pulse geben die Hall-Sensoren pro Umdrehung?

Über den SPEED-Pin des BLDC-8015A Drivers können die Umdrehungen gemessen werden. Die einzelnen Pulse müssen mit einem 5k Pullup Widerstand verbunden werden (siehe Schaltplan „Measure BLDC Speed“), sonst erkennt es der Arduino nicht. Dann im Arduino mit einem Interrupt Event verbinden und die einzelnen Pulse zählen.

```
| attachInterrupt(digitalPinToInterrupt(WIND_SPEED_OUT_PIN),countRPM,RISING);
```

```
void countRPM(){

    pulses++;

    currentTime2 = millis(); // get ms

    if((currentTime2 - lastTime2) >= sek60){
        Serial.print("Rotations: ");
        Serial.print(pulses/WIND_PULSES_PER_ROTATION);
        Serial.print("\n\n");
        lastTime2 = currentTime2;
    }

}
```

Anschließend ein Stück Klebeband an die Achse (damit man die 360° erkennt) kleben und ihn manuell um 360° drehen. Jetzt hat man die Anzahl der Pulse pro Umdrehung und kann so die Umdrehung bestimmen und steuern. Der Wert ist in der Variable „WIND_PULSES_PER_ROTATION“ abgelegt.

Jetzt wurde mit der Interruptfunktion nach jeder Minute eine Anzahl der Rotationen ausgegeben.

Mit diesen Methoden wurde zurückgerechnet, welchen Wert die „map“ funktion ausgibt. Dadurch ist der Wert 4800 zustandegekommen.

```
writeWindRPM = map(WIND_RPM,0,4800,0,255);
```

$$\frac{WIND_RPM}{4800} = \frac{Pulsausgabe}{255}$$

Jetzt müssen wir die 4800 anpassen, da nicht unsere *WIND_RPM*, sondern die *gemessenen_RPM* mit dieser *Pulsausgabe* erreicht wurde. Die Pulsausgabe muss evtl. noch gerundet werden, da der Arduino keinen 20,15 Puls mit PWM ausgeben kann (schätze ich).

$$\frac{gemessene_RPM}{y} = \frac{Pulsausgabe}{255}$$

Nach y gelöst, kommt bei mir ein Wert von 4728 heraus. Deshalb heißt die neue funktion so.

```
writeWindRPM = map(WIND_RPM,0,4728,0,255);
```

Der Code zum Zählen der RPM und anpassen der Daten ist in gekennzeichneten Kommentarblöcken sortiert:

```
/* For Measuring the RPM Pulses
```

```
attachInterrupt(digitalPinToInterrupt(WIND_SPEED_OUT_PIN),countRPM,RISING);  
lastTimeRPM = millis();  
sek60 = 60000;  
*/
```

Starten des BLDC Motor

Im Setup-Code als letztes, also bevor er in die Loop geht, wird die Richtung des BLDC Motors eingestellt und er mit den vorher berechneten RPM gestartet.

```
digitalWrite(WIND_ROTATION_DIRECTION_PIN,HIGH); // set direction of winding motor. HIGH = clockwise  
analogWrite(WIND_AVI_CONTROL_PIN, writeWindRPM); // set Motor speed
```

Stepper Motor

Der Stepper Motor hängt an der X-Achse und ist für die Drahtführung zuständig. Er wird als STEP oder Ähnlich bezeichnet.

Kalibrierung des Stepper Motors

Pro Umdrehung des Wind-Motors muss sich die X-Achse exakt um den Drahtdurchmesser weiterbewegen. Damit ein Draht schön neben dem anderen liegt. In welcher Zeit müssen also Pulse gesendet werden, der Stepper Motor die richtige Geschwindigkeit hat?

Das berechnet die Funktion.

```
double calculateStepSetup()
```

In einer Umdrehung des Wind Motors möchte ich eine Drahtspur legen.

$$Wire_Per_Rot_W = 1$$

Die WIND_RPM müssen jetzt noch in $Rot/\mu s$ umgerechnet werden.

$$Rot_W_Per_usek = \frac{WIND_RPM}{60 \frac{s}{min} * 1000 \frac{ms}{s} * 1000 \frac{\mu s}{ms}}$$

Die Zeit (in Mikrosekunden), in der ein Draht gelegt werden muss, ist also:

$$Wire_Per_usek = Wire_Per_Rot_W * Rot_W_Per_usek$$

Jetzt müssen wir noch berechnen, wie viele Pulse die X-Achse fahren muss, damit in dieser Zeit genau die Strecke einer Drahtbahn (= Drahtdurchmesser) zurückgelegt wird.

Wenn sich die X-Achse um 360° dreht, legt die Schiene eine Distanz von SPINDLE_PITCH zurück. Zuerst brauchen wir den Umdrehungswinkel, um die Distanz von einem Draht zurückzulegen. Dieser lässt sich wie folgt berechnen:

$$Rot_X_Per_Wire [1] = \frac{WIRE_DIAMETER [\mu m]}{SPINDLE_PITCH [\mu m]}$$

Die Zeit, die X-Achse für eine Umdrehung braucht, lässt sich somit wie folgt berechnen:

$$usek_Per_Rot_X = \frac{1}{Rot_X_Per_Wire * Wire_Per_usek}$$

Auf dem Driver DM542 steht in der pulse/rev Tabelle, wie viele Pulse man senden muss, um eine Umdrehung zu erzielen. Je nach Schalterstellung. Dieser Wert wird in der Konstanten STEP_MOTOR_DRIVER_PULSE_PER_REV gespeichert. Damit lässt sich dann auch die Zeit pro Pulse berechnen.

$$usek_Per_Pulse = \frac{usek_Per_Rot_X}{STEP_MOTOR_DRIVER_PULSE_PER_REV}$$

Achtung! Die berechnete Zeit usek_Per_Pulse ist die Zeit für einen On-Off-Cycle (also HIGH- und LOW-Puls zusammengerechnet), deshalb wird die berechnete Zeit nochmal halbiert.

$$usek_Per_Pulse_Change = \frac{usek_Per_Pulse}{2}$$

Um Rundungs und Overflow Fehler zu vermeiden bzw. auszuschließen, wird die Berechnung im Code in eine Formel zusammengefasst, dann gekürzt. Der Arduino berechnet also nur eine Codezeile.

In der Dokumentation des Motortreibers „DM542T.pdf“ (nicht derselbe Treiber, aber nahezu identisch) steht unter „9. Sequence Chart of Control Signals“, dass die Pulsweite eines HIGH- bzw. LOW-Pulses nicht 2.5 μs unterschreiten darf. Dieser Wert wird in der Konstante STEP_DRIVER_MIN_USEK_PER_PULSE_CHANGE gespeichert und nach Berechnung der Pulszeit überprüft.

```
//TODO if its below min, throw error
if(usek_Per_Pulse_Change < STEP_DRIVER_MIN_USEK_PER_PULSE_CHANGE){
    return 0;
}
```

Programmablauf

Setup

Am Anfang werden alle Pins initialisiert. Anschließend werden beide Motoren initialisiert und die Anzahl der Reihen berechnet.

```
// unclear if enable pin is needed or not connected, documentation is not clear
// machine works this way...
digitalWrite(STEP_ENABLE_PIN, LOW); //so x-axis can move

usekPerPulseX = ceil(calculateStepSetup()); //calculate PulseTime for X-Axis, round up, so no error
XAxisRows = round(calculateXAxisRowCount()); // only does full rows, rounds RowCount
distanceX = PICKUP_WINDING_HEIGHT; // distance the Motor has to move per Row

// start turing motor with specific rpm
writeWindRPM = map(WIND_RPM,0,4728,0,255);

bool start = false;
```

Danach wird über den Serial Monitor die jeweiligen Daten für die aktuelle Wicklung ausgegeben. Die X-Achse startet dann dem Homing Vorgang und bewegt sich zur Startposition.

```
moveToStart(); // startup program
```

Jetzt wird gewartet, bis die Start-Taste gedrückt wird.

```
// == Start Logic ==
Serial.println("Press Start Button to Start Winding");

// wait to press start button
do{
  StartProgram = digitalRead(START_PROGRAM_PIN);
  delay(50); // 50ms
}
while(StartProgram == LOW);

Serial.println("Starting winding...");
```

Jetzt wird die richtige Richtung der X-Achse eingestellt und der Wind-Motor gestartet.

Loop

Solange noch Reihen zu fahren sind, fährt er eine Reihe

```
posX = moveDistance(posX,distanceX); // move forward or backwards, depending
```

Wobei nach jeder Reihe die Richtung geändert wird, sodass er immer hin und her fährt.

```
distanceX = distanceX * (-1);
```

Sind keine Reihen mehr zu fahren, so wird ausgegeben dass er fertig ist und das Programm beendet. Um das Programm erneut zu starten und den nächsten Tonabnehmer zu wickeln, am Arduino die RESET Taste drücken.

```
analogWrite(WIND_AVI_CONTROL_PIN, 0);
Serial.println("Completed Winding.\n\n");
delay(5000);
exit(0);
```

Emergency Stop

Der Emergency Stop wird durch ein Interrupt behandelt. Hier wird die Funktion securityStop() aufgerufen, welche das Programm beendet.

Funktionen im Überblick

calculateXAxisRowCount

Hier wird die Anzahl der Reihen berechnet, die die X-Achse fahren muss.

$$\text{Number_of_Rows} * \text{Number_of_Windings_Per_Row} = \text{Number_of_Windings}$$

$$\text{Number_of_Windings_Per_Row} = \frac{\text{Pickup_Winding_Height}}{\text{Wire_Diameter}}$$

Jetzt werden beide Formeln nach der Anzahl der Reihen umgestellt und der Wert zurückgegeben.

calculateStepsToDistance

Die Anzahl der Schritte die der Schrittmotor machen muss, um die gewünschte Distanz zu gehen. Da der Schrittmotor nur eine gerade Anzahl an Schritten gehen kann, wird das Ergebnis gerundet.

moveDistance

Die X-Achse fährt die angegebene Distanz. Ist die Distanz negativ fährt er zum Motor hin, ist sie Positiv fährt er vom Motor weg. $pulse = 2 * steps$, da für jeden Schritt ein HIGH und LOW Puls gesendet werden muss. Anschließend fährt der Motor in der Geschwindigkeit, die vorher in

```
calculateStepSetup()
```

berechnet wurde.

setMoveDirection

Setzt die Richtung der X-Achse.

moveStepperPerTimeStep()

Implementiert die Logik, um die X-Achse einen Schritt zu bewegen. Nachdem überprüft wurde, ob genügend Zeit vergangen ist, wie in

| *calculateStepSetup()*

berechnet wurde, wird der nächste Puls (je nachdem ob HIGH oder LOW) geschrieben. Ist das passiert gibt die Funktion 1 zurück. Dadurch lässt sich berechnen, wie viele Pulse schon gesendet worden sind und wie weit die X-Achse gefahren ist.

Es wird nicht überprüft, dass genau die Zeit vergeht, bis der nächste Schritt kommt. Es wäre möglich, dass durch anderen Code die Funktion deutlich später erneut aufgerufen wird. Die Bedingung ist zwar erfüllt, aber der Draht würde nicht mehr synchron zum Wickelmotor aufgetragen werden. Dadurch hätte man weniger als einen Draht pro Umdrehung.

goToZero

Setzt die Richtung automatisch in Richtung Schrittmotor und bewegt sich dann solange auf ihn zu, bis der Endstopp ausgelöst wird.

moveToStart

Die Top Level Funktion, um die X-Achse zur Startposition zu bewegen. Erst wird

| *goToZero();*

aufgerufen, anschließend zur Startposition bewegt. Die Startposition ist die Position, wo die erste Drahtbahn auf dem Tonabnehmer liegt. Außerdem gibt die Funktion Statusupdates im Serial Monitor aus.

Konfigurationseinstellungen – Config.h

Die Config Datei ist in drei Bereiche unterteilt. Die User Calibrated Settings, die Hardware Configuration und das X-Axis-Protocol.

User Calibrated Settings

Das sind Einstellungen, die sich von Wicklung zu Wicklung unterscheiden können.

Wenn zum Beispiel ein anderer Tonabnehmer gewickelt werden soll oder anderer Draht verwendet wird oder schneller bzw. langsamer gewickelt.

Hardware Configuration

Das sind Einstellungen, die man einmal macht, damit die Maschine funktioniert.

z.B. alle Pins Ports oder die *STEP_MOTOR_DRIVER_PULSE_PER_REV* die durch den Schrittmotortreiber vorgegeben werden.

X-Axis Protocol

Die minimale Zeit, die eine Pegeländerung (HIGH zu LOW oder LOW zu HIGH) haben darf. Vorgegeben vom Schrittmotortreiber.

Hardware

Die Komponenten, die verwendet wurden können in der [Einkaufsliste](#) gefunden werden.

Der generelle Aufbau ist im 3D Modell zu sehen, mit den einzelnen Teilen, die 3D gedruckt wurden. Der Schaltplan ist in einem extra PDF zu sehen.

Die Drehplatte, die im 3D-Modell, bzw. auf den Bildern zu sehen ist, wird nicht genutzt, da die Drahtspindel auf dem Boden steht. Der Draht lässt sich so leichter abwickeln. Der Endstop wurde auch entgegen des 3D-Modells mit doppelseitigem Klebeband befestigt.

Für einzelne Feineinstellungen bei der Hardware, ist das Kapitel [Hardwarekonfiguration](#) gedacht.

